



2



5



7.5 sp

TD

كلية الهندسة المعلوماتية

السنة الأولى

## المصفوفات في Java

د. ماهر صارم



RB Informatcs; 6/4/2026

محتوى مجاني غير مخصص للبيع التجاري

البرمجة 2

### تعريف المصفوفة

يمكن تعريف المصفوفة بطريقتين:

**إما:** `int [ ] b = {1,2,3,4,5};`

بالإمكان تعريفها بشكل مباشر مع قيمها (مثل c++) : `int [ ] a = {1,2,3,4};`

**أو:** `int [ ] a = new int [10];` (الطريقة الأكثر استعمالاً في java)

**ملاحظة:** في جافا لا يمكن تعريف المصفوفة بدون تحديد حجمها

في جافا هنالك خاصية تساعد في معرفة عدد العناصر للمصفوفة وهي: `a.length` وتستخدم في الشرط في حلقات for لإدخال أو طباعة عناصر المصفوفة.

مثال:

```
int a[] = {1,2,3,4,5};
for(int i=0;i<a.length;i++){
    System.out.println(a[i]);
}
```

### خاصية for each

في جافا يوجد خاصية في حلقات التكرار for تستخدم مع المصفوفات، وهي for each وهي من الشكل:

```
int a[] = {0,2,7,5};
for(int x : a){
    System.out.println(x);
}
```



في حلقة for المتغير  $x$  يعبر عن عناصر المصفوفة  $a$  ويجب أن يكون معرف بنفس نوع تعريف المصفوفة. الحلقة تطبق من أجل كل عنصر  $x$  من المصفوفة  $a$  أي أن  $x$  يعبر عن عناصر المصفوفة وتتغير قيمته بعد كل دورة حسب ترتيب عناصر المصفوفة.

#### الفرق بين for و for each للمصفوفات:

- في حلقة for لدي تحكم كامل بالمصفوفة، أي يمكنني تحديد البداية أو النهاية أو حتى كمية الزيادة على المضمون (i) index أو النقصان منه.
- في حلقة for each سيتم المرور على كل العناصر بالمصفوفة بالترتيب (غالباً يستعمل لإدخال أو طباعة جميع عناصر المصفوفة).

#### تطبيق:

```

1- import java.util.Scanner;
2- public class Main {
3-     public static void main(String[] args) {
4-         int [] a = new int[10];
5-         Scanner in = new Scanner(System.in);
6-         for(int x:a) {
7-             x=in.nextInt();
8-         }
9-         for(int i=0;i<a.length;i++)
10-            {System.out.println(a[i]);
11-            }
12-     }
13- }

```

#### شرح الكود:

6 -> 8: حلقة for each لإدخال عناصر المصفوفة (يمكن استعمال for)

9 -> 11: حلقة for لطباعة عناصر المصفوفة (يمكن استعمال حلقة for each)

a.length -> يجلب عدد عناصر المصفوفة.

## الـ ArrayList:

- هي مصفوفة ديناميكية قابلة للزيادة أو النقصان في الحجم تلقائياً، وتتميز ب:
- نوع البيانات: يمكن أن تخزن أنواع متنوعة من القيم (أكثر من نوع متغير).
- طريقة التخزين: تعامل الـ ArrayList ككائن (object) وتعتمد في عملها على المؤشرات.
- توزيع الذاكرة: يتم حجز عناصرها في أماكن متفرقة في الذاكرة، لكنها تترتب بواسطة المؤشرات حيث يشير كل عنصر إلى مكان العنصر الذي يليه.

## مثال عن كيفية عمل الـ ArrayList :

```
import java.util.ArrayList;
public class Main {
    public static void main(String[]
args) {
        ArrayList al = new ArrayList();
        al.add("A");
        al.add("B");
        al.remove("A");
        al.size();
        al.remove(1);
        system.out.println(al);
        al.add(1, "C");
        System.out.println(al.get(0));
    }
}
```

(طريقة إضافة عنصر لمصفوفة)

(طريقة حذف عنصر من المصفوفة)

(طريقة لمعرفة حجم المصفوفة)

(حذف عنصر لكن باستخدام الـ index)

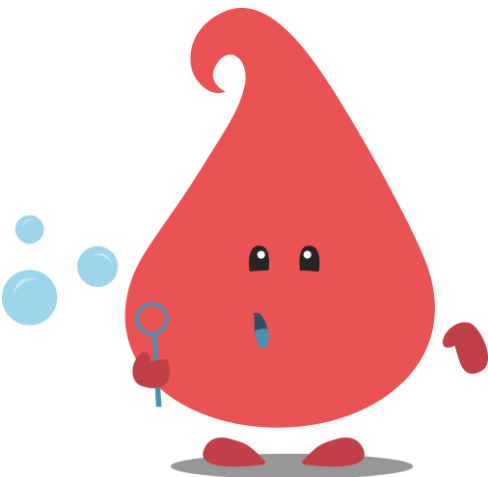
(طباعة المصفوفة كاملة دون حلقة for)

(طريقة إضافة عنصر لمصفوفة بحسب الـ index)

(طباعة عنصر حسب الـ index)

## ملاحظات:

- بالإضافة (add): يضيف على نهاية المصفوفة.
- بالحذف (remove): يبدأ بالبحث من أول عنصر ويستمر حتى يصل إلى أول عنصر يطابق المطلوب فيقوم بحذفه.





إضافة آريسيزية:

لماذا لم نضع أقواس تعريف المصفوفة المعتادة [ ] ؟:

السبب هو أن ArrayList في java ليست مصفوفة عادية بل هي كائن (object) من فئة (class) جاهزة في اللغة. نحن نستخدم () لأننا نقوم بإنشاء كائن من فئة برمجية وليس مجرد حجز مساحة تخزين ثابتة كما تفعل الأقواس [].

## طريقة تعريف التوابع في java

```
public class Main {
    public static void
    main(String[] args) {
    }
    public static int f( );{
    }
}
```

الوسيط

اسم التابع

نوع الإرجاع

**نلاحظ أن** تعريف التابع يتم ضمن class Main قبل أو بعد void main على عكس c++, حيث أنه في c++ تعريف التابع حصراً قبل (main).

**سؤال:** هل يمكن للتوابع إرجاع مصفوفة؟

**الجواب:** نعم يمكنها، سيتم توضيح ذلك في التطبيق القادم.

If you never try, you will  
never know.



شرح الكود:

```

1- public class Main {
2-     public static void
main(String[] args) {
3-         double[] a = {3,8,9,2,7};
4-         double[] reverse;
5-         reverse = revArray(a);
6-         for (double x : reverse){
7-             System.out.println(x);
8-         }
9-     }
10- public static double[]
revArray(double[] a) {
11-     double temp;
12-     for(int i=0;i<a.length/2;i++)
{
13-         temp = a[i];
14-         a[i] = a[a.length-i-1];
15-         a[a.length-i-1]=temp;
16-     }
17-     return a;
18- }
}

```

10: تابع يقوم بعكس عناصر المصفوفة وإعادة المصفوفة المعكوسة.

11: تعريف متغير double يقوم بتخزين قيمة مؤقتة لتبديلها بمكان آخر في المصفوفة.

12: حلقة for تتوقف عند نصف المصفوفة لأننا سنبدل طرفي المصفوفة ببعضهما (لو كان عدد العناصر فردي لن نبدل العنصر في المنتصف، ولو كان زوجي فلن يكون هناك عنصر في المنتصف بل عنصرين وسيتم تبديلهم).

13: نضع العنصر الحالي في المتغير المؤقت temp

14: نأخذ العنصر المقابل له في النهاية ونخزنه في البداية.

15: نخزن قيمة المتحول المؤقت في العنصر (الذي تم تبديل قيمه) في النهاية.

توضيح آلية عمل دورات الحلقة:

- الدورة الأولى: نبدل العنصر الأول (3) بالعنصر الأخير (7) فتصبح المصفوفة كما يلي: {7,8,9,2,3}
- الدورة الثانية: نبدل العنصر الثاني (8) بالعنصر قبل الأخير (2) فتصبح المصفوفة: {7,2,9,8,3}
- عند المنتصف يبقى الرقم (9) في مكانه لأنه في المنتصف ولا يوجد ما يقابله لتبديله.

😊 انتهت المحاضرة الثانية